End-to-End System & Workflow Documentation

Embedded-C IoT Protocol Laboratory and Portable Firmware Architecture

Detailed Functional Diagrams, Parameter Catalogs, and Execution Workflows for
PC Simulation and STM32 NUCLEO-L452RE-P + Quectel EC200U-CN

IoT Firmware Engineering Team

October 2026

Abstract

This document provides an exhaustive, end-to-end technical reference for the portable IoT firmware project and protocol evaluation laboratory. The system evaluates two standard telemetry and device-management architectures: **MQTT** (Eclipse Paho Embedded-C over TCP/TLS) and **LwM2M** (Eclipse Wakaama over CoAP/UDP/DTLS). It operates across two primary environments: a high-fidelity **PC Simulation Environment** equipped with a C-based web dashboard host (`lab.exe`) and real local/public brokers, and an embedded **Target Hardware Environment** deploying the exact same core firmware stack to an **ST NUCLEO-L452RE-P** microcontroller interfaced to a **Quectel EC200U-CN** LTE Cat-1 cellular modem. This specification documents every functional block, message flow sequence, modem AT state machine, hardware wiring interface, compilation define, and runtime parameter.

Contents

1 Executive Summary and Project Scope **3**

2 Functional Architecture and System Diagrams **3**
- 2.1 PC Simulation Architecture . . . 3
- 2.2 Embedded Target Architecture (STM32 Nucleo + EC200U) . . . 3
- 2.3 Firmware Layering and Modularity . . . 3

3 Hardware Architecture and Interface Specifications **3**
- 3.1 Electrical Wiring and Pinout Mapping . . . 3
- 3.2 Microcontroller Clocks and Peripherals . . . 6
- 3.3 Memory Allocations and Static Budgets . . . 6

4 Modem Interface and AT State Machine **6**
- 4.1 Modem Bring-Up Sequence . . . 6
- 4.2 AT Socket Lifecycle (Direct Push Mode) . . . 7

5 Protocol Workflows and Sequence Diagrams **7**
- 5.1 MQTT End-to-End Session Workflow . . . 7
- 5.2 LwM2M End-to-End Session Workflow . . . 7
- 5.3 Simulated Over-The-Air (OTA) Update Workflow . . . 7

6 Comprehensive Parameter Catalog **8**
- 6.1 Build-Time CMake and Preprocessor Definitions . . . 8
- 6.2 Hardware Configuration Defines (`hw_config.h`) . . . 9
- 6.3 Command-Line and Dashboard Runtime Options . . . 9

7 Step-by-Step Operational Workflows **11**
7.1 PC Simulation Workflow 11
7.2 STM32 Target Hardware Workflow 11

8 Measurement Methodology and Experimental Results **12**
8.1 Latency Calculation Formulas 12
8.2 Experimental Performance Comparison 12
8.3 Key Technical Conclusions 12

1 Executive Summary and Project Scope

Modern IoT devices face trade-offs between protocol simplicity (MQTT) and rich device-management capabilities (LwM2M), as well as complex network characteristics across low-bandwidth cellular networks. This project was developed to provide:

1. **High-Fidelity Evaluation Without Hardware Dependencies:** Developers can measure latency, on-the-wire byte overhead, connection setup overhead, and network loss tolerance on a host PC before flashing hardware.
2. **100% Portable Protocol Logic:** The protocol engines (`mqtt_client.c`, `lwm2m_client.c`), telemetry encoding (`telemetry.c`), filtering (`sensor.c`), and transport security layers (`link.c`) are strictly written in C11 and compiled identically on both PC and microcontroller.
3. **Drop-in Hardware Portability:** Operating system and peripheral dependencies are decoupled via a thin Platform Abstraction Layer (`pal.h` / `port.h`). On PC, it interfaces with Winsock/POSIX; on the STM32 target, `pal_ec200u.c` directs data streams through the Quectel EC200U cellular modem's AT command socket stack.
4. **Transparent, Empirical Measurements:** Every byte crossing the wire is accounted for (including TCP/IP overhead and TLS/DTLS record envelopes). No simulated metrics are fabricated; missing responses are strictly flagged as failures.

2 Functional Architecture and System Diagrams

The system is deployed across two architectural variants: the PC host simulation and the real embedded hardware testbed.

2.1 PC Simulation Architecture

In the PC simulation, the complete system runs locally as independent, concurrent C processes communicating over loopback sockets.

2.2 Embedded Target Architecture (STM32 Nucleo + EC200U)

In the hardware configuration, the PC simulated processes are replaced by the **ST NUCLEO-L452RE-P** microcontroller running `pg_firmware.elf`. The PC continues to run `lab.exe`, which communicates bidirectionally over the ST-LINK USB Virtual COM Port (USART2) at 921,600 baud. The STM32 drives the **Quectel EC200U-CN** modem over USART1 at 115,200 baud through a 1.8V bidirectional level shifter.

2.3 Firmware Layering and Modularity

The firmware follows strict layered isolation. High-level layers only invoke functions declared directly below them:

3 Hardware Architecture and Interface Specifications

3.1 Electrical Wiring and Pinout Mapping

The target hardware configuration couples an ST NUCLEO-L452RE-P development board to a Quectel EC200U-CN breakout board.

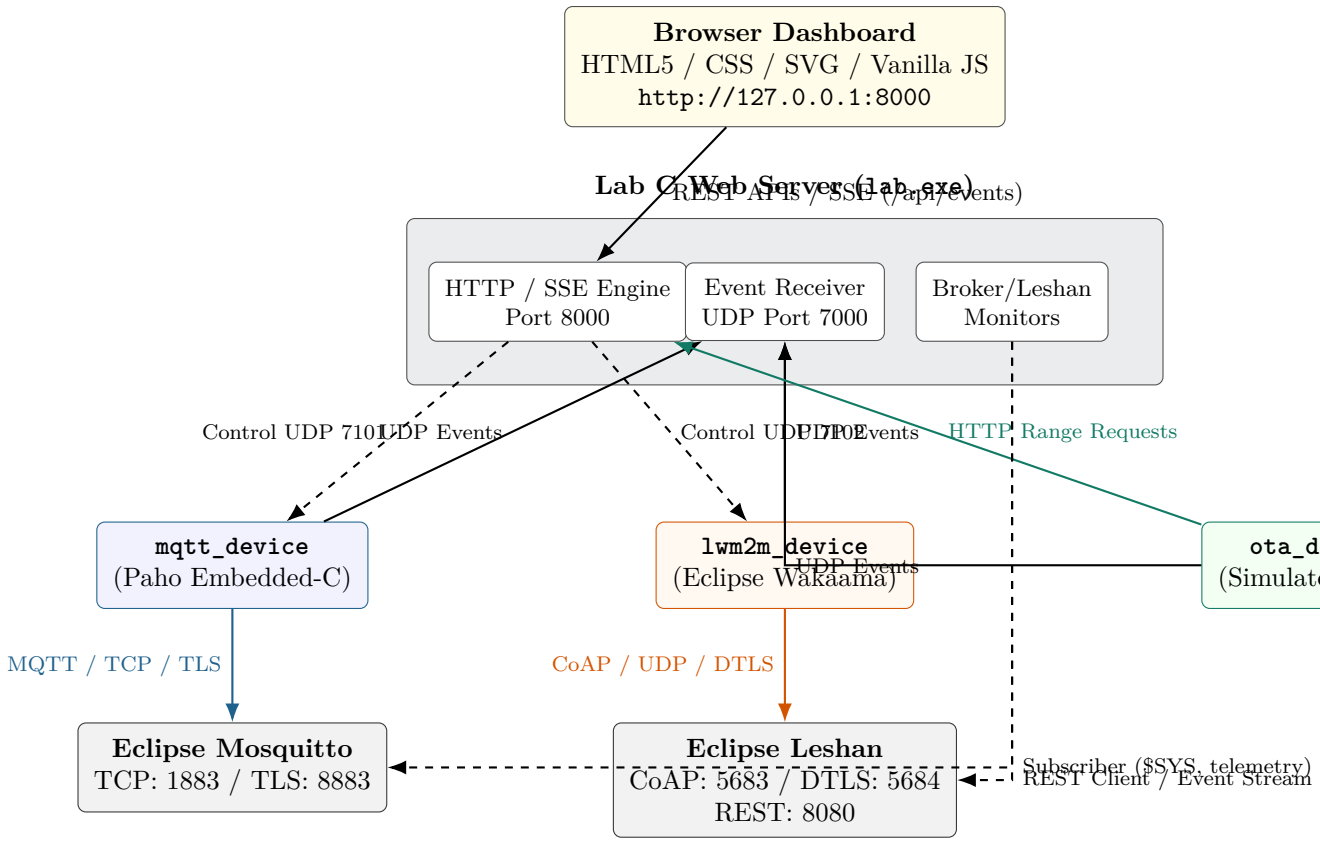


Figure 1: PC Laboratory Execution and Communication Architecture.

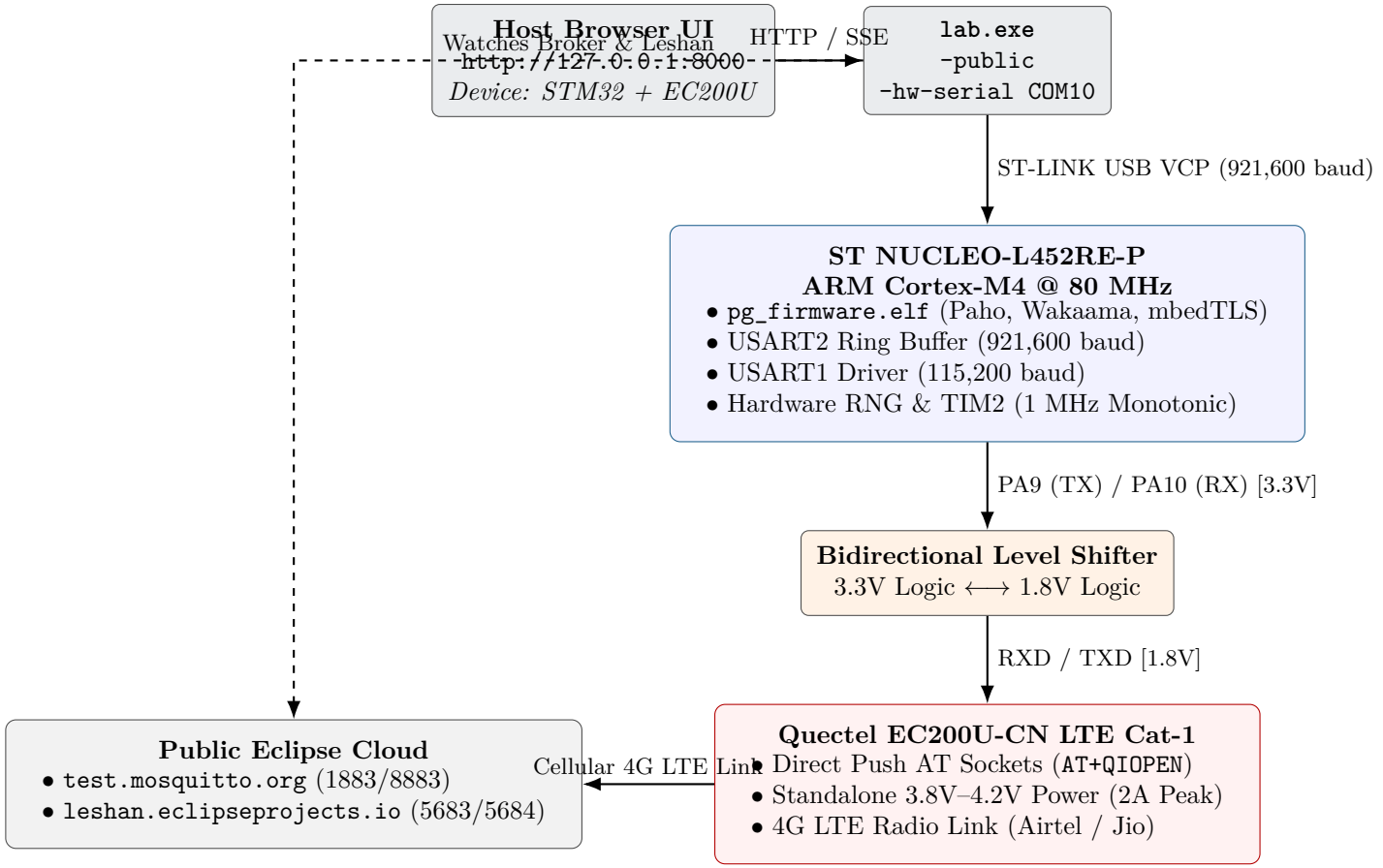


Figure 2: Embedded Target (STM32 + EC200U) End-to-End Testbed Architecture.

Layer	Component Files	Functional Responsibility
Application	`device.c`, `telemetry.c`, `sensor.c`	High-level state machine, sampling scheduling, JSON telemetry formatting, and Exponential Moving Average (EMA) filtering ($\alpha = 0.3$).
Protocol	`protocol_manager.c`, `mqtt_client.c`, `lwm2m_client.c`	Backend selection, connection state monitoring, exponential reconnect backoff, Paho MQTT packet parsing, and Wakaama CoAP engine.
Security	`tls.c`, `dtls.c`, `tls_common.c`	TLS 1.3/1.2 client over TCP, DTLS 1.2 PSK over UDP, mbedTLS CTR-DRBG entropy seeding, and certificate chain verification.
Network	`network.c`, `socket.c`, `link.c`, `netsim.c`	Generic byte transport interface (`fw_transport_t`), socket accounting, and synthetic network latency/loss delay lines.
HAL	`timer.c`, `storage.c`, `power.c`	Wrap-safe deadline software timers, CRC32-validated key-value persistent storage (`"FWST"`), and battery voltage scaling.
Platform (PAL)	`pal.c` (PC), `pal_ec200u.c` (Board), `port_stm32.c`	Operating system abstraction: monotonic microsecond clocks, sleep, and physical/AT socket dispatch.

Table 1: Modular Firmware Layer Structure.

Nucleo Pin	STM32 Alternate Fn	EC200U Pin	Logic Level	Wiring & Circuit Instructions
PA9 (D8)	`USART1_TX` (AF7)	`RXD`	1.8V Domain	**Level Shift Required:** Connect Nucleo 3.3V TX to high side; shift down to 1.8V modem RX.
PA10 (D2)	`USART1_RX` (AF7)	`TXD`	1.8V Domain	**Level Shift Required:** Connect modem 1.8V TX to low side; shift up to 3.3V Nucleo RX.
PA0 (A0)	GPIO Output	`PWRKEY`	Open Drain	Pulsed active LOW for 1,250 ms only if the modem fails to respond to initial `AT`.
PA1 (A1)	GPIO Output	`RESET`	Active Low	Optional hardware reset pin (reserved).
PA2 / PA3	`USART2_TX / RX`	ST-LINK MCU	3.3V CMOS	Virtual COM Port (USB) routed directly to Host PC at 921,600 baud.
GND	Ground	`GND`	0V Reference	Must be tied together across Nucleo, Level Shifter, and Modem board.
External DC	N/A	`VBAT`	3.8V–4.2V	**Mandatory 2A Peak Supply:** Cellular bursts exceed USB current limits.

Table 2: Electrical Wiring Matrix between NUCLEO-L452RE-P and Quectel EC200U-CN.

3.2 Microcontroller Clocks and Peripherals

- **System Clock (SYSCLK):** Configured to 80 MHz using the internal Multi-Speed Oscillator (MSI @ 4 MHz) driven through Phase-Locked Loop (PLL) parameters: $M = 1, N = 40, R = 2$.
- **True Random Number Generator (RNG):** Sourced via the dedicated 48 MHz internal oscillator (`HSI48`), supplying hardware entropy to mbedTLS via `mbedtls_hardware_poll()`.
- **High-Resolution Monotonic Timer (TIM2):** Configured as a 32-bit hardware timer running at 1 MHz (prescaler = 79), providing microsecond timestamps (`pal_now_us()`) that do not drift.
- **Cellular UART (USART1):** Configured to 115,200 baud, 8N1. Serviced by an interrupt-driven 8,192-byte ring buffer in SRAM to prevent overrun during cellular data bursts.
- **Lab Link UART (USART2):** Configured to 921,600 baud, 8N1. High throughput ensures 1 KB JSON event strings transfer in under 11 ms, preserving real-time protocol timings.

3.3 Memory Allocations and Static Budgets

The NUCLEO-L452RE-P provides 512 KiB Flash and 160 KiB SRAM. To fit the dual protocol stacks, buffers were optimized:

- **Flash Utilization:** Approximately 290 KiB (including mbedTLS crypto, Paho, Wakaama, and embedded PEM certificates).
- **SRAM Static Footprint:** Approximately 80 KiB allocated to static ring buffers, leaving 80 KiB for RTOS/C-runtime stacks and dynamic heap.
- **Buffer Trimming:** The PC's 20 KiB `MQTT_BUF_SIZE` was reduced to 4,096 bytes; the Wakaama RX buffer is set to 2,048 bytes; the network queue depth is set to 32 slots.

4 Modem Interface and AT State Machine

On the STM32 target, `ec200u.c` implements a robust finite state machine to manage power, cellular network registration, and socket handling.

4.1 Modem Bring-Up Sequence

1. **AT Handshake:** Transmits `AT\r\n` every 500 ms. If no response is received within 3 seconds, `PA0` (PWRKEY) is pulled active-low for 1,250 ms to cycle power, followed by a 30-second boot poll.
2. **Base Configuration:**
   - `ATE0`: Disables command echo.
   - `AT+CMEE=2`: Enables verbose error reporting.
   - `AT+QURCCFG="urcport","uart1"`: Routes Unsolicited Result Codes (URCs) to USART1 instead of USB.
   - `AT+QICFG="dataformat",0,0`: Sets raw binary mode for socket transfers.
   - `AT+QISDE=0`: Disables echoed send data during socket writes.
3. **SIM and Radio Verification:** Polls `AT+CPIN?` until `READY` (timeout 10 s). Polls `AT+CEREG?` until status indicates Home Network (1) or Roaming (5) (timeout 180 s).
4. **PDP Context Activation:** Configures APN via `AT+QICSGP=1,1,"<APN>","","",1`. Activates context via `AT+QIACT=1` (timeout 150 s). Extracts assigned local IPv4 address.

4.2 AT Socket Lifecycle (Direct Push Mode)

To preserve compatibility with the PC's BSD socket interface, sockets are configured in Direct Push mode:

- **Connect:** Executes `AT+QIOPEN=1,<id>,"TCP"|"UDP","<host>",<port>,0,1`. Waits for URC `+QIOPEN: <id>,0`.
- **Transmit:** Executes `AT+QISEND=<id>,<len>`. Waits for the prompt character `>`. Emits binary payload. Awaits confirmation `SEND OK`. Packets exceeding 1,460 bytes are segmented automatically.
- **Receive:** When data arrives, the modem emits `+QIURC: "recv",<id>,<len>` followed immediately by the raw byte stream. The interrupt service routine reads these bytes directly from the UART into the socket's internal FIFO ring buffer.
- **Close:** Executes `AT+QICLOSE=<id>,10`.

5 Protocol Workflows and Sequence Diagrams

5.1 MQTT End-to-End Session Workflow

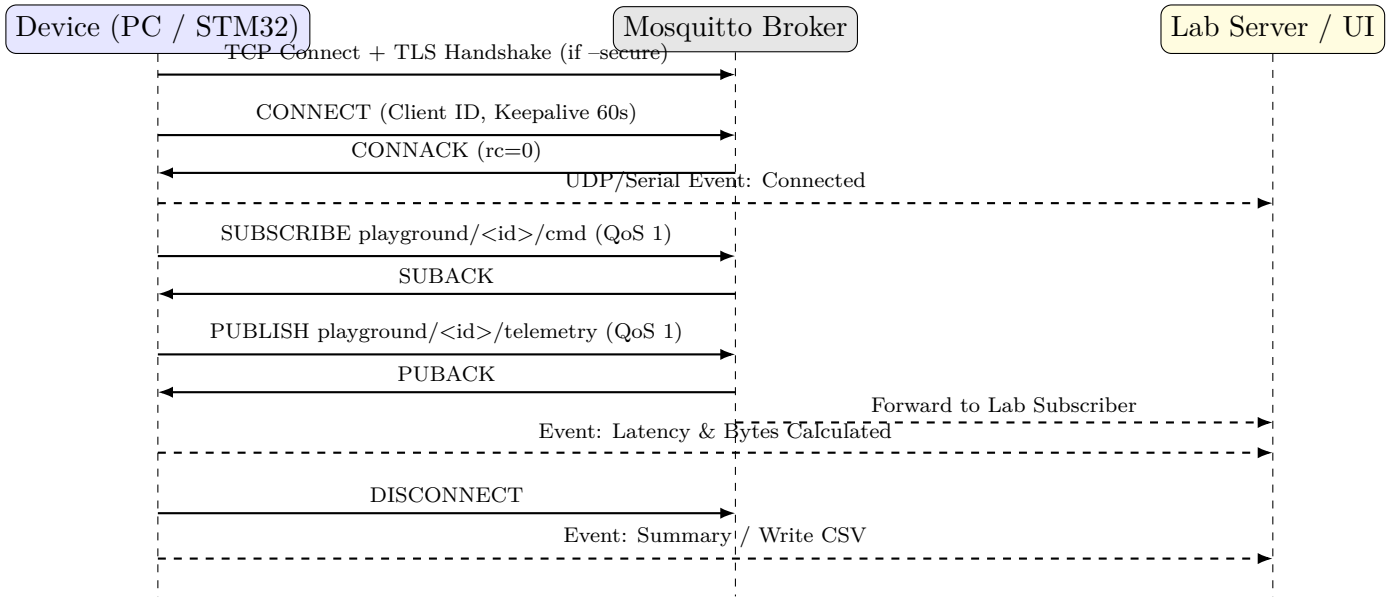


Figure 3: MQTT Telemetry Session Sequence.

5.2 LwM2M End-to-End Session Workflow

5.3 Simulated Over-The-Air (OTA) Update Workflow

The OTA mechanism models firmware distribution without requiring custom flash layouts:

1. **Manifest Retrieval:** The device executes an HTTP GET to `/ota/manifest?image=<name>`, receiving a JSON object containing target image size and SHA-256 digest.
2. **Chunked Range Downloads:** The client issues segmented HTTP Range requests (`Range: bytes=offset-(offset+chunk-1)`), saving chunks to `results/ota_storage/slot_b.bin`. Failed chunks are retried up to 3 times.
3. **Integrity Verification:** The device computes the SHA-256 hash over the downloaded file. If it matches the manifest, `active_version.txt` is updated; otherwise, the download is aborted.

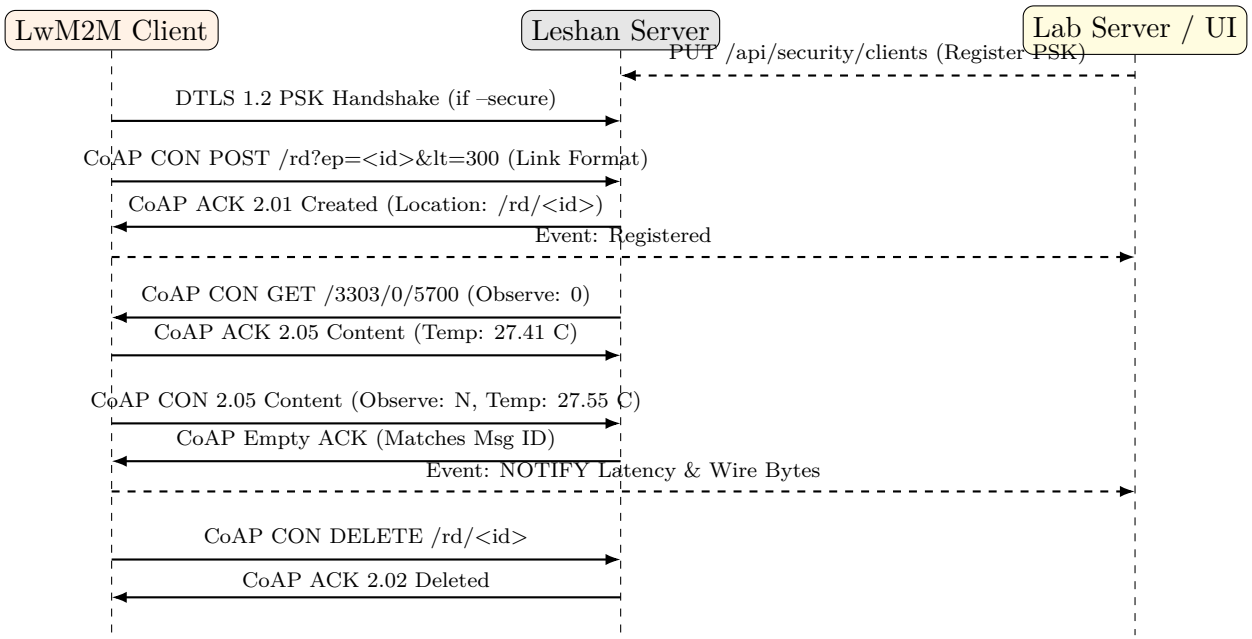


Figure 4: LwM2M Registration, Observation, and Notification Sequence.

6 Comprehensive Parameter Catalog

This catalog defines every parameter in the codebase, detailing its default value, valid range, and configuration point.

6.1 Build-Time CMake and Preprocessor Definitions

Parameter Macro	Default	Range / Options	Description & Configuration Point
`FW_PLATFORM`	`"pc"`	`"pc"`, `"stm32"`	CMake cache variable: `-DFW_PLATFORM=pc`. Selects platform directory.
`FW_PROTOCOL`	`0` (MQTT)	`0` (MQTT), `1` (LwM2M)	Set in `fw_config.h`. Selects default compile-time protocol.
`FW_USE_MBEDTLS`	`ON`	`ON`, `OFF`	CMake option. Controls mbedTLS library compilation.
`FW_USE_PAHO`	`ON`	`ON`, `OFF`	CMake option. Enables Paho Embedded-C packet parser.
`FW_USE_WAKAAMA`	`ON`	`ON`, `OFF`	CMake option. Enables Eclipse Wakaama LwM2M client.
`FW_MQTT_HOST`	`"test.mosquitto.org"`	Hostname / IPv4	Build-time default MQTT broker address.
`FW_MQTT_USE_TLS`	`0`	`0` (Plain), `1` (TLS)	Build-time flag toggling TLS encryption for MQTT.
`FW_MQTT_PORT`	1883 / 8883	1–65535	TCP port (1883 if plain, 8883 if TLS).

Parameter Macro	Default	Range / Options	Description & Configuration Point
FW_MQTT_KEEPALIVE_S	60	5–3600 seconds	MQTT PINGREQ keep-alive interval.
FW_MQTT_BUF_SIZE	1024 / 4096	Bytes	Serialization buffer size for Paho packets.
FW_MQTT_MAX_SUBS	4	1–16	Maximum allowable active MQTT subscriptions.
FW_TOPIC_MAX	128	Bytes	Maximum length of MQTT topic strings.
FW_LWM2M_HOST	"leshan..."	Hostname / IPv4	Build-time default LwM2M server address.
FW_LWM2M_USE_DTLS	0	0 (Plain), 1 (DTLS)	Build-time flag toggling DTLS-PSK for LwM2M.
FW_LWM2M_PORT	5683 / 5684	1–65535	UDP port (5683 if plain, 5684 if DTLS).
FW_LWM2M_BUF_SIZE	1280	Bytes	CoAP message buffer allocation.
FW_TLS_ALLOW_INSECURE	0	0 (Verify), 1 (Allow)	Bypasses TLS CA certificate validation (development only).
FW_NET_TIMEOUT_MS	5000	Milliseconds	Maximum blocking timeout on socket operations.
FW_TELEMETRY_INTERVAL_MS	10000	1000–86400000 ms	Nominal sensor telemetry transmission period.
FW_LOOP_BUDGET_MS	100	Milliseconds	Network polling budget inside device_step().
FW_RECONNECT_MIN_MS	1000	Milliseconds	Initial exponential reconnect backoff delay.
FW_RECONNECT_MAX_MS	60000	Milliseconds	Maximum reconnect backoff ceiling.
FW_BATTERY_LOW_MV	3300	Millivolts	Low-battery threshold triggering rate reduction.
FW_LOW_BATTERY_FACTOR	4	Multiplier	Interval multiplier when battery voltage drops below threshold.

Table 3: Compile-Time Configuration Parameters.

6.2 Hardware Configuration Defines (hw_config.h)

6.3 Command-Line and Dashboard Runtime Options

- **Common Device Parameters:**
  - -host <str>: Target broker or server hostname.

Macro	Default	Scope	Functional Purpose
MODEM_APN	"airtelgprs.com"	Cellular PDP	APN string ("airtelgprs.com", "jionet", "bsnlnet").
MODEM_PDP_CONTEXT	1	EC200U Context	PDP context profile number passed to AT+QIACT.
MODEM_BAUD	115200	USART1	Baud rate for modem UART interface.
MODEM_REG_TIMEOUT_MS	180000	LTE Registration	Maximum wait time for AT+CEREG? attachment.
MODEM_PWRKEY_PORT	GPIOA	Pin Mapping	GPIO port controlling modem PWRKEY FET.
MODEM_PWRKEY_PIN	GPIO_PIN_0	Pin Mapping	GPIO pin for PWRKEY signal.
MODEM_PWRKEY_PULSE_MS	1250	Power Timing	Duration PWRKEY is held low to boot modem.
VCP_BAUD	921600	USART2	Baud rate for ST-LINK USB Virtual COM port.

Table 4: Target Hardware Configuration Defines.

  - -port <num>: Target server port.
  - -id <str>: Client endpoint identity (e.g. mqtt-dev-1).
  - -interval <dur>: Transmission cadence (10s, 500ms).
  - -count <num>: Message limit before automatic exit (0 = infinite).
  - -payload <bytes>: Enforces exact payload size by padding JSON.
  - -seed <num>: Random seed ensuring reproducible sensor walks.
  - -secure: Enables TLS (MQTT) or DTLS-PSK (LwM2M).
  - -profile <name>: Network condition profile: excellent (0ms, 0%), good (50ms, 1%), poor (250ms, 5%), very-poor (1000ms, 10%).
  - -latency <ms> / -loss <pct>: Custom latency and drop rates.
  - -rto <ms>: TCP retransmission timeout used by netsim.c (default 200 ms).
  - -lab <host:port>: Target for JSON event datagrams (default 127.0.0.1:7000).

- **MQTT-Specific Options:**
  - -qos <0|1|2>: Quality of Service level (default 1).
  - -no-probe: In QoS 0, disables trailing PINGREQ delivery confirmation.
  - -keepalive <sec>: MQTT ping interval (default 60 s).

- **LwM2M-Specific Options:**
  - -lifetime <sec>: Registration lifetime on server (default 300 s).
  - -notify-non: Transmits notifications as Non-confirmable (NON).
  - -ack-timeout <dur>: CoAP retransmission timeout (default 2 s).

- **Lab Server Options (lab.exe):**
  - -http-port <num>: Web dashboard port (default 8000).
  - -events-port <num>: UDP event listening port (default 7000).
  - -hw-serial <port>: Serial port connecting to STM32 (e.g. COM10).
  - -public: Configures all entities to use Eclipse public cloud sandboxes.

7 Step-by-Step Operational Workflows

7.1 PC Simulation Workflow

1. **Server Provisioning:**

```
# Option A: Start local servers using portable scripts
.\scripts\get-servers.ps1
.\scripts\start-mosquitto.ps1 -Port 21883 -TlsPort 28883
.\scripts\start-leshan.ps1
```

2. **PC Build:**

```
cd R:\Revin\Firmware\firmware\mqtt-lwm2m-playground
cmake -S . -B build
cmake --build build
ctest --test-dir build
```

3. **Launch Lab Host:**

```
.\build\lab.exe --mqtt-port 21883 --mqtts-port 28883
# Or using public servers:
.\build\lab.exe --public
```

4. **Dashboard Interaction:** Open `http://127.0.0.1:8000`. Verify server status indicators show `ONLINE`. Select the MQTT or LwM2M tab, adjust parameters, and click **START**.

7.2 STM32 Target Hardware Workflow

1. **Wiring and Electrical Inspection:** Ensure level shifters protect the 1.8V modem lines. Verify the modem is supplied by an external 2A power source.

2. **Compilation via STM32CubeCLT Toolchain:**

```
cd R:\Revin\Firmware\firmware\mqtt-lwm2m-playground
$clt = "R:\STM_driver_VS\STM32CubeCLT_1.22.0"
$hw  = "R:/Revin/Firmware/firmware/mqtt-lwm2m-playground/hardware/stm321452-ec200u"

& "$clt\CMake\bin\cmake.exe" -S $hw -B build-stm32 -G Ninja `
  "-DCMAKE_MAKE_PROGRAM=$clt/Ninja/bin/ninja.exe" `
  "-DCMAKE_TOOLCHAIN_FILE=$hw/cmake/arm-none-eabi.cmake" `
  "-DHW_DEFINES=MODEM_APN=\"airtelgprs.com\""

& "$clt\CMake\bin\cmake.exe" --build build-stm32
```

3. **Flashing Target Microcontroller:**

```
& "$clt\CMake\bin\cmake.exe" --build build-stm32 --target flash
```

4. **Modem Attachment Verification:** Observe the on-board green LED (`LD4`). Solid ON indicates cellular IP acquisition; fast blinking (250 ms toggle) indicates registration failure.

5. **Execute Serial Bridge:**

```
.\build\lab.exe --public --hw-serial COM10
```

6. **Remote Hardware Control:** On the web dashboard, toggle the device dropdown to **Device: STM32 + EC200U**. The Board pill will turn green (`READY`). Trigger protocol runs to stream telemetry over cellular LTE.

8 Measurement Methodology and Experimental Results

8.1 Latency Calculation Formulas

All timestamps are recorded in microseconds immediately prior to socket submission, ensuring network model delays are captured:

- **MQTT QoS 1:** Latency = t(PUBACK) − t(PUBLISH).
- **MQTT QoS 2:** Latency = t(PUBCOMP) − t(PUBLISH).
- **MQTT QoS 0 Probe:** Latency = t(PINGRESP) − t(PUBLISH) (measured via trailing 4-byte `PINGREQ`).
- **LwM2M CON:** Latency = t(CoAP ACK) − t(CON NOTIFY).

8.2 Experimental Performance Comparison

Configuration	Median Latency	P95 Latency	Wire Bytes/Msg	Handshake Bytes
MQTT QoS 1 (Plain TCP)	0.38 ms	0.71 ms	158.7 B	—
MQTT QoS 1 (TLS 1.3)	0.42 ms	0.85 ms	207.0 B	2,812 B
LwM2M CON (Plain UDP)	1.17 ms	2.74 ms	36.8 B	—
LwM2M CON (DTLS 1.2 PSK)	1.25 ms	2.90 ms	94.8 B	601 B
Loss 5% – MQTT QoS 1	0.37 ms	208 ms	165.2 B	—
Loss 5% – LwM2M CON	1.07 ms	2,769 ms	42.1 B	—

Table 5: Benchmark Results on PC Loopback Environment (Natural Payloads, Interval 0.2 s).

8.3 Key Technical Conclusions

1. **Bandwidth Overhead:** LwM2M with compact SenML/CBOR encoding requires approximately 75% fewer bytes per transmission than MQTT transmitting JSON over TCP (36.8 B vs 158.7 B).
2. **Packet Loss Resilience:** Under lossy network profiles, MQTT maintains lower tail latency (P95 ≈ 200 ms) due to kernel-level TCP fast retransmission. In contrast, CoAP's application-level exponential backoff doubles retransmission timers (2–3 s), leading to P95 latencies in the multi-second range.
3. **Cellular Serial Bottlenecks:** On the STM32 + EC200U target, a 1 KB message incurs ≈90 ms of latency purely crossing the 115,200 baud UART link before reaching the cellular modem transmitter.
4. **Cryptographic Processing Overhead:** Performing TLS/DTLS asymmetric handshakes on the Cortex-M4 CPU core requires ≈1 second due to mathematical computation, compared to ≈12 ms on the PC host.